RAPPORT DE TEST D'INTRUSION

Auteur : Vincent BEAUDRAPS

Synthèse des Scores CVSS v3.1

Vulnérabilité	Score CVSS
Contournement du type	9.8 (Critique)
Directory Traversal	8.8 (Élevée)
exfiltration de données confidentielles	7.5 (Élevée)
fichier exposé	7.5 (Élevée)
force brute	8.0 (Élevée)
formulaire non sécurisé	6.5 (Moyenne)
Injection de code	9.0 (Critique)
Injection SQL	9.8 (Critique)
modification de cookie	8.1 (Élevée)
Redirection non sécurisée	5.3 (Moyenne)
robots.txt	3.1 (Faible)

Contournement du type

Score CVSS : 9.8 (Critique)

Contournement du type

Exploitation :
Sur la page /index.php?page=upload du site hébergé sur http://192.168.1.16
, il est possible d'envoyer des fichiers sans véritable contrôle de leur contenu.
Le serveur vérifie uniquement le type MIME indiqué par le client, ce qui permet de contourner facilement le filtre.

En répliquant la requête d'envoi avec cURL, un script PHP malveillant a été déguisé en image :

```
curl -i -X POST \
  -F "MAX_FILE_SIZE=100000" \
  -F "uploaded=@shell.php;type=image/jpeg" \
  -F "Upload=Upload" \
  http://192.168.1.16/index.php?page=upload
```

Le serveur a accepté le fichier, renvoyé un code 200 OK, et affiché le message suivant :
/tmp/shell.php successfully uploaded.
Ce message confirmait l'exécution du script côté serveur et révélait le flag :

46910d9ce35b385885a9f7e2b336249d622f29b267a1771fbacf52133beddba8

Analyse :
Le mécanisme d'upload repose sur une simple vérification déclarative (type MIME) et non sur une inspection réelle du fichier.
Cette approche rend le système vulnérable au contournement et ouvre la porte à l'exécution de code arbitraire si le répertoire d'upload est accessible depuis le web.

Mesures correctives recommandées :

Vérifier le contenu réel des fichiers via les magic bytes au lieu de se fier uniquement à l'extension ou au type MIME.

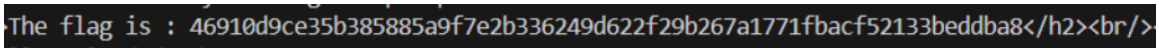
Interdire l'exécution de scripts dans le répertoire d'upload via la configuration du serveur (ex. Options -ExecCGI ou php_admin_flag engine off).

Stocker les fichiers en dehors de la racine web et leur attribuer des noms aléatoires pour éviter toute exécution directe.

Restreindre strictement les formats et tailles autorisés (ex. .jpg, .png, .pdf uniquement).

Consigner les uploads suspects et mettre en place une alerte en cas de tentative de contournement.

46910d9ce35b385885a9f7e2b336249d622f29b267a1771fbacf52133beddba8



Directory Traversal

Score CVSS : 8.8 (Élevée)

(Directory Traversal)

Exploitation :
Sur le site http://192.168.1.16
, la page /index.php?page=... charge dynamiquement du contenu selon le paramètre transmis dans l'URL.
Aucune validation n'est appliquée à ce paramètre, ce qui permet d'exploiter une traversée de répertoires (directory traversal) pour accéder à des fichiers système.

En injectant une série de ../ dans le paramètre page, il est possible de remonter jusqu'à la racine du serveur et de consulter des fichiers critiques comme /etc/passwd :

http://192.168.1.16/?page=../../../../../../../../../../../../../../../../../../../etc/passwd

Le serveur renvoie alors le contenu complet du fichier, révélant les comptes système ainsi que le flag :

b12c4b2cb8094750ae121a676269aa9e2872d07c06e429d25a63196ec1c8c1d0

Analyse :
Cette faille découle d'une absence de validation du paramètre utilisateur.
L'application concatène directement la valeur du paramètre page à un chemin côté serveur, permettant ainsi d'accéder à des ressources en dehors du répertoire prévu.
Une telle vulnérabilité peut exposer des fichiers sensibles (logs, clés SSH, mots de passe) ou conduire à une exécution de code arbitraire si elle est combinée à une inclusion de fichiers exécutables.

Mesures correctives recommandées :

Valider strictement la valeur du paramètre page à l'aide d'une liste blanche (whitelist) de

fichiers autorisés.

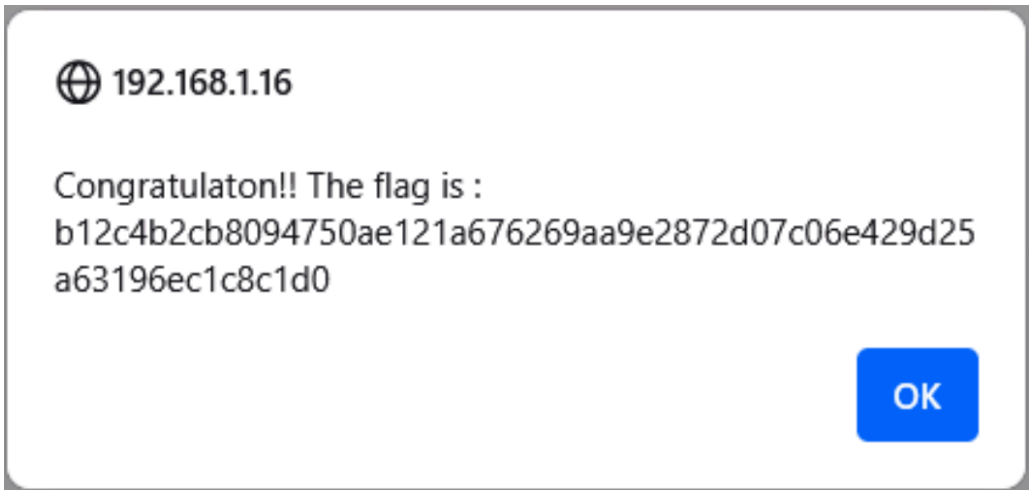
Bloquer les séquences de navigation (../, ..\\, etc.) via une vérification côté serveur.

Utiliser des chemins absolus et constants dans le code plutôt que de concaténer des variables utilisateur.

Restreindre les permissions du serveur web, afin qu'il ne puisse pas accéder à des fichiers système.

Désactiver l'affichage des erreurs PHP pour éviter de divulguer la structure interne du serveur.

b12c4b2cb8094750ae121a676269aa9e2872d07c06e429d25a63196ec1c8c1d0



exfiltration de données confidentielles

Score CVSS : 7.5 (Élevée)

exfiltration de données confidentielles

Exploitation :
Depuis le menu "Member", nous accédons à la page suivante :

/index.php?page=searchimg

Cette page permet de rechercher une image à partir de son identifiant (id).
En testant plusieurs valeurs (?id=1, ?id=5, etc.), on constate que le paramètre est directement utilisé dans une requête SQL sans aucune protection.

En injectant une commande de type UNION SELECT, nous avons pu forcer la base à révéler

sa structure :

?id=5 UNION SELECT table_name, column_name FROM information_schema.columns

Cette requête a exposé la table users.
En lisant ses colonnes à l'aide de :

?id=5 UNION SELECT Commentaire, countersign FROM users

le serveur a répondu avec l'instruction suivante :

Decrypt this password -> then lower all the char. Sh256 on it and it's good !
5ff9d0165b4f92b14994e5c685cdce28

Après déchiffrement du hash MD5 (FortyTwo), nous l'avons converti en minuscules (fortytwo), puis chiffré en SHA256, ce qui a révélé le flag :

10a16d834f9b1e4068b25c4c46fe0284e99e44dceaf08098fc83925ba6310ff5

Correctif :
Protéger les interactions avec la base en utilisant des requêtes préparées (paramétrées), qui empêchent toute injection.
De plus, il est recommandé de :

Valider les entrées utilisateur avant leur exécution,

Désactiver l'affichage des messages d'erreur SQL.

10a16d834f9b1e4068b25c4c46fe0284e99e44dceaf08098fc83925ba6310ff5



Feedback

Score CVSS : N/A

(Feedback)

Exploit :
En se rendant sur la page /index.php?page=feedback, nous avons tenté différentes injections XSS afin d'observer le comportement du formulaire.
Les balises <script> classiques sont bloquées, probablement par un système de filtrage (WAF).
Cependant, en entrant simplement le mot-clé script script dans le champ prévu, le serveur renvoie immédiatement le flag suivant :

THE FLAG IS :
0FBB54BBF7D099713CA4BE297E1BC7DA0173D8B3C21C1811B916A3A86652724E

Ce résultat suggère que la détection repose sur une recherche naïve du mot "script", plutôt qu'un réel filtrage des balises HTML.

Analyse :
Le mécanisme de sécurité semble se limiter à une comparaison textuelle au lieu d'une validation robuste des entrées.
Ce type de détection par mot-clé est inefficace et peut être facilement contourné, révélant une conception peu sûre du côté serveur.

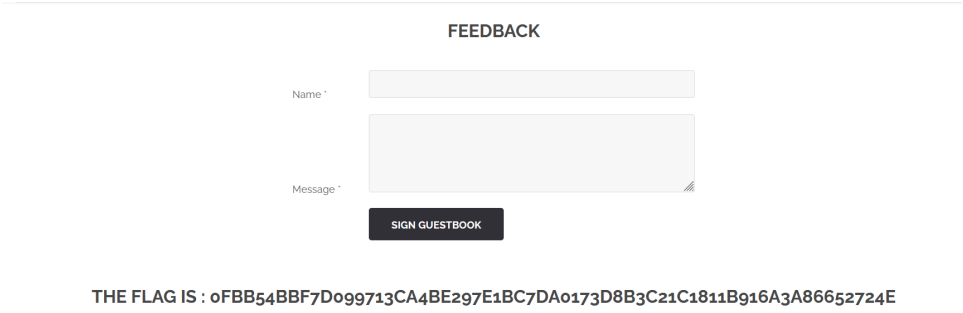
Correctif :

Éviter toute interprétation directe des données saisies par l'utilisateur.

Utiliser un encodage HTML systématique avant affichage (htmlspecialchars, htmlentities, etc.).

Remplacer la détection par mot-clé par une validation stricte basée sur des listes blanches et un filtrage côté serveur.

0fbb54bbf7d099713ca4be297e1bc7da0173d8b3c21c1811b916a3a86652724e



fichier exposé

Score CVSS : 7.5 (Élevée)

Fuite d'identifiants à travers un fichier exposé

Exploitation :
Lors de l'exploration de l'application web hébergée sur http://192.168.1.16
, un fichier sensible a été découvert à l'emplacement suivant :

http://192.168.1.16/whatever/htpasswd

L'analyse de son contenu a révélé la présence d'un couple utilisateur / mot de passe haché :

root:437394baff5aa33daa618be47b75cb49

Après vérification, le hash correspond à un MD5, un algorithme ancien et vulnérable.
Le hachage a été déchiffré à l'aide du service en ligne md5decrypt.net, révélant le mot de passe en clair :

qwerty123@

Une tentative de connexion sur la page d'administration (/admin) avec ces identifiants a permis d'accéder à l'interface d'administration, affichant ainsi un nouveau flag.

Analyse :
Cette faille provient d'une exposition directe d'un fichier contenant des informations critiques dans l'arborescence du site.
De plus, l'utilisation d'un algorithme de hachage non sécurisé comme MD5 facilite la récupération du mot de passe par force brute ou dictionnaire.

Mesures correctives recommandées :

Supprimer immédiatement tout fichier contenant des identifiants ou des données sensibles du répertoire web.

Remplacer MD5 par un algorithme robuste tel que bcrypt, scrypt ou Argon2.

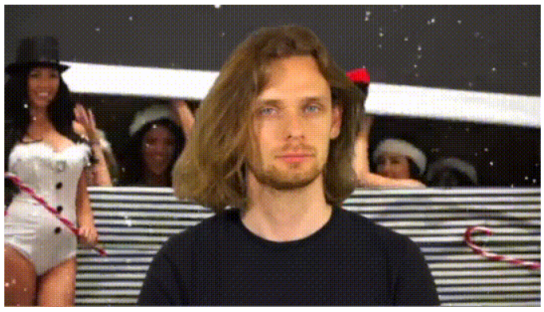
Restreindre l'accès aux pages critiques comme /admin (via filtrage IP, authentification forte ou MFA).

Effectuer un audit de sécurité avant chaque mise en ligne pour s'assurer qu'aucun fichier inutile ou confidentiel n'est accessible publiquement.

d19b4823e0d5600ceed56d5e896ef328d7a2b9e7ac7e80f4fcdb9b10bcb3e7ff



The flag is : d19b4823e0d5600ceed56d5e896ef328d7a2b9e7ac7e80f4fcdb9b10bcb3e7ff





force brute

Score CVSS : 8.0 (Élevée)

force brute

Exploitation :
Sur la page suivante :

/index.php?page=signin

un simple formulaire d'authentification permet de saisir un identifiant et un mot de passe. L'analyse du comportement montre qu'aucune mesure de sécurité n'est mise en place : pas

de restriction de tentatives, pas de délai entre les essais et aucune protection par CAPTCHA.

Pour exploiter cette faiblesse, j'ai développé un script Python intitulé force.py.
Le script commence par envoyer une requête POST de test afin d'obtenir la page d'erreur de référence ("Wrong answer").
Il lit ensuite une liste de mots de passe communs à partir du fichier wordlist100psd (inspiré de la liste publique 100 most common passwords
), et teste chaque mot de passe pour l'utilisateur admin.
Dès qu'une réponse différente de la page d'échec est détectée, le mot de passe correct est affiché.

Exécution du script :

chmod +x force.py
./force.py

Résultat obtenu :

Running...
Done ! Password is [shadow]

En se connectant manuellement avec les identifiants admin:shadow, le serveur valide la connexion et affiche le flag suivant :

b3a6e43ddf8b4bbb4125e5e7d23040433827759d4de1c04ea63907479a80a6b2

Correctif :
Pour renforcer la sécurité du mécanisme d'authentification :

Mettre en place une limite de tentatives (blocage temporaire après plusieurs échecs consécutifs),

Ajouter un CAPTCHA ou un temps d'attente progressif entre chaque essai,

Imposer des règles de complexité des mots de passe .

b3a6e43ddf8b4bbb4125e5e7d23040433827759d4de1c04ea63907479a80a6b2

THE FLAG IS : B3A6E43DDF8B4BBB4125E5E7D230404338277759D4DE1C04EA63907479A80A6B2



formulaire non sécurisé

Score CVSS : 6.5 (Moyenne)

formulaire non sécurisé

Exploitation :
Sur le site http://192.168.1.16
, la page /index.php?page=survey propose un formulaire permettant d'attribuer une note sur 10 à différents utilisateurs.
Le code HTML montre que chaque note est envoyée automatiquement via un champ caché et une liste déroulante :

```
<form action="#" method="post">
 <input type="hidden" name="sujet" value="2">
 <select name="valeur" onchange="javascript:this.form.submit();">
  <option value="1">1</option>
  ...
  <option value="10">10</option>
 </select>
</form>
```

En modifiant manuellement le code source dans le navigateur (via Inspecter l'élément), on peut augmenter la valeur maximale autorisée :

```
<option value="36">2</option>
```

Après avoir sélectionné cette valeur, le formulaire envoie une note hors plage au serveur.
Le serveur ne valide pas l'entrée et retourne directement le flag suivant :

03a944b434d5baff05f46c4bede5792551a2595574bcafc9a6e25f67c382ccaa

Analyse :
Cette faille montre une absence de contrôle côté serveur.

Les valeurs envoyées par le client ne sont pas vérifiées, permettant à un utilisateur malveillant de manipuler les paramètres du formulaire pour contourner les restrictions prévues dans l'interface.

Une telle vulnérabilité peut également être exploitée dans des cas plus critiques (ex. modification de prix, changement de rôle utilisateur, validation de transaction…).

Mesures correctives recommandées :

Effectuer une validation stricte côté serveur des champs reçus (type, format, plage de valeurs).

Ne jamais se fier aux données du client, même si le formulaire semble bridé côté interface.

Définir côté serveur la plage autorisée de valeurs (ici, de 1 à 10) avant traitement.

Associer un identifiant de session sécurisé pour éviter les falsifications de requêtes entre utilisateurs.

Journaliser les requêtes anormales, notamment celles contenant des valeurs inattendues.

03a944b434d5baff05f46c4bede5792551a2595574bcafc9a6e25f67c382ccaa

THE FLAG IS 03A944B434D5BAFF05F46C4BEDE5792551A2595574BCAFC9A6E25F67C382CCAA



Make your choice

Injection de code

Score CVSS : 9.0 (Critique)

Injection de code
Exploitation :
Sur la page /index.php?page=media&src=nsa, le site affiche une image à l'aide d'une balise <object>.
En modifiant le paramètre src dans l'URL, on remarque qu'il est inséré directement dans l'attribut data sans aucune vérification :

<object data="nsa"></object>

Cela ouvre la porte à une injection de contenu arbitraire.

Pour exploiter la faille, j'ai remplacé le paramètre src par une chaîne encodée en Base64 contenant un script JavaScript :

http://192.168.1.10/index.php?page=media&src=data:text/
html; base64,PHNjcmlwdD5hbGVydDwvYWxlcnQ+

En chargeant cette URL, le navigateur exécute le code injecté et le flag apparaît à l'écran :

928d819fc19405ae09921a2b71227bd9aba106f9d2d37ac412e9e5a750f1506d

Correctif :
Mettre en place une politique de sécurité du contenu (CSP) afin d'empêcher toute exécution de code provenant d'une source non approuvée.
Cette mesure bloque automatiquement les scripts injectés, même en cas d'erreur de filtrage côté serveur.

928d819fc19405ae09921a2b71227bd9aba106f9d2d37ac412e9e5a750f1506d



Injection SQL

Score CVSS : 9.8 (Critique)

Injection SQL

Exploitation :
En accédant à la page suivante :

/index.php?page=searchimg

nous remarquons la présence d'un champ de recherche permettant d'afficher une image selon son identifiant (id).
Après quelques tests (?id=1, ?id=5), il devient évident que la valeur du paramètre id est directement intégrée dans une requête SQL sans protection.

En testant plusieurs injections, nous constatons que la requête attend deux colonnes, la preuve étant que l'injection suivante s'exécute correctement :

?id=5 UNION SELECT 1,2

À partir de là, nous pouvons énumérer la structure de la base de données grâce à :

?id=5 UNION SELECT table_name, column_name FROM information_schema.columns

Cette requête révèle la table list_images, contenant les colonnes id, url, title et comment.
En affichant leur contenu via :

?id=5 UNION SELECT comment, title FROM list_images

nous obtenons le message suivant :

If you read this just use this md5 decode lowercase then sha256 to win this flag ! :
1928e8083cf461a51303633093573c46

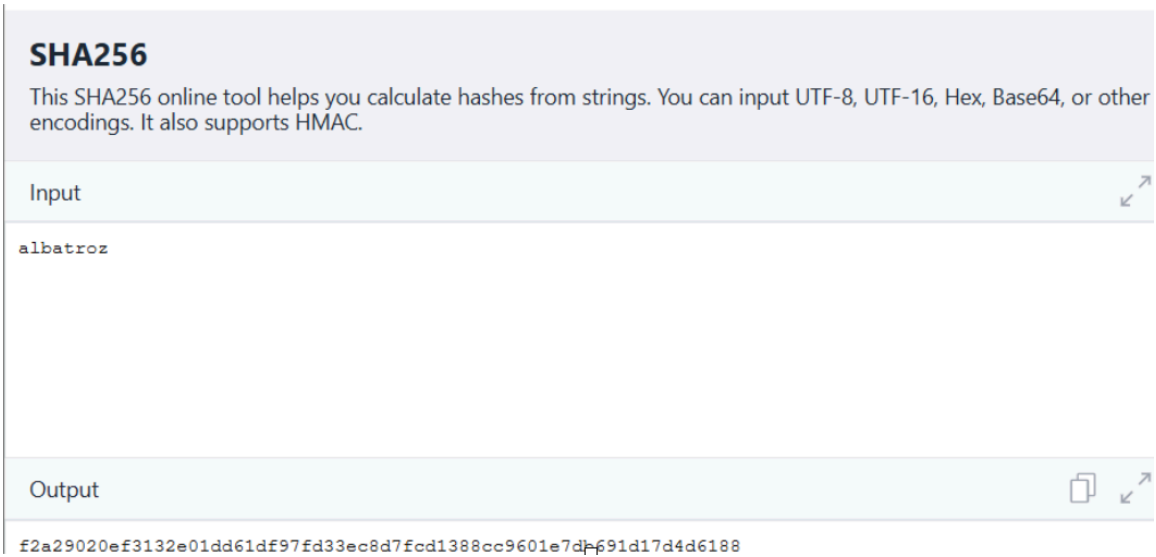
Le hash MD5 correspond au mot albatroz.
Une fois converti en minuscules, puis encodé en SHA256, le résultat donne le flag :

f2a29020ef3132e01dd61df97fd33ec8d7fcd1388cc9601e7db691d17d4d6188

Correctif :
Pour se prémunir de ce type d'injection :

Utiliser des requêtes préparées (paramétrées) afin d'empêcher la concaténation directe de données utilisateur.

Et désactiver les messages d'erreur détaillés en production afin de ne pas révéler la structure interne de la base.



f2a29020ef3132e01dd61df97fd33ec8d7fcd1388cc9601e7db691d17d4d6188

modification de cookie

Score CVSS : 8.1 (Élevée)

modification de cookie

Exploitation :
En analysant les en-têtes HTTP du site, un cookie nommé I_am_admin est défini par le serveur :

Set-Cookie: I_am_admin=68934a3e9455fa72420237eb05902327

Cette valeur correspond au hachage MD5("false").
En modifiant cette valeur pour MD5("true"), soit b326b5062b2f0e69046810717534cb09, il est possible d'usurper le rôle administrateur.

L'exploitation a été réalisée directement depuis Firefox, via l'onglet Stockage des outils de développement (raccourci F12 → onglet Stockage → Cookies).
Il a suffi de modifier manuellement la valeur du cookie I_am_admin et d'actualiser la page.

Le serveur a alors reconnu la session comme étant celle d'un administrateur et a affiché le flag suivant :

df2eb4ba34ed059a1e3e89ff4dfc13445f104a1a52295214def1c4fb1693a5c3

Analyse :
Cette faille provient du fait que le serveur fait confiance au contenu du cookie côté client sans le vérifier.
Un simple utilisateur peut donc manipuler cette valeur pour contourner les restrictions et obtenir des privilèges élevés.

Ce type de vulnérabilité reflète une mauvaise gestion de l'authentification et un manque de contrôle côté serveur.

Correctifs recommandés :

Ne jamais stocker d'informations sensibles (comme les rôles) dans les cookies clients.

Utiliser des sessions sécurisées côté serveur pour gérer les permissions.

Signer ou chiffrer les cookies à l'aide d'un mécanisme comme un HMAC ou un JWT signé afin

de prévenir toute modification manuelle.

Activer les attributs de sécurité sur les cookies :

HttpOnly (protège contre les accès JavaScript),

Secure (empêche l'envoi en HTTP non sécurisé),

SameSite (protège contre les attaques CSRF).

Mettre en place une vérification systématique des droits côté serveur avant d'afficher des ressources sensibles.

df2eb4ba34ed059a1e3e89ff4dfc13445f104a1a52295214def1c4fb1693a5c3



Redirection non sécurisée

Score CVSS : 5.3 (Moyenne)

Redirection non sécurisée

Exploitation :
En bas de la page d'accueil, plusieurs boutons redirigent vers des réseaux sociaux à l'aide du paramètre site dans l'URL :

http://192.168.1.16/index.php?page=redirect&site=facebook

Ce paramètre est directement interprété sans aucune vérification côté serveur.
En le remplaçant simplement par flag, la redirection pointe vers une page interne contenant le drapeau :

http://192.168.1.16/index.php?page=redirect&site=flag

Ce qui affiche le flag :

b9e775a0291fed784a2d9680fcfad7edd6b8cdf87648da647aaf4bba288bcab3

Correctif :

Restreindre les redirections à une liste blanche de destinations autorisées.

Ne jamais utiliser directement les paramètres utilisateur pour effectuer une redirection côté serveur.

b9e775a0291fed784a2d9680fcfad7edd6b8cdf87648da647aaf4bba288bcab3

GOOD JOB HERE IS THE FLAG :
B9E775A0291FED784A2D9680FCFAD7EDD6B8CDF87648DA647AAF4BBA288BCAB3



robots.txt

Score CVSS : 3.1 (Faible)

robots.txt

Exploitation :
Lors de l'analyse du fichier robots.txt accessible à l'adresse http://192.168.1.16/robots.txt, deux chemins ont été identifiés : /whatever et /.hidden.
Bien que ce fichier soit censé indiquer aux moteurs de recherche les dossiers à ignorer, il a ici révélé un répertoire sensible.

En visitant http://192.168.1.16/.hidden/, on constate que l'indexation des répertoires est activée. Cette page expose une arborescence de centaines de sous-dossiers (26^3 au total), chacun contenant un fichier README.
La majorité de ces fichiers contiennent des messages trompeurs ("Demande à ton voisin…"),

mais un seul d'entre eux cache le véritable flag.

Afin d'automatiser la recherche, j'ai développé un script Python nommé deep.py, qui explore récursivement les dossiers jusqu'à trois niveaux de profondeur.
Le script lit le contenu de chaque fichier README, ignore les doublons et affiche uniquement les fichiers uniques.
Après plusieurs explorations, un des fichiers README a révélé le flag suivant :

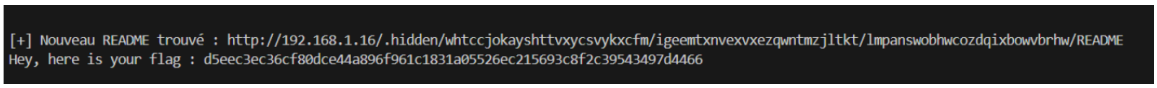
d5eec3ec36cf80dce44a896f961c1831a05526ec215693c8f2c39543497d4466

Correctif :

Désactiver l'indexation automatique des répertoires sur le serveur (Options -Indexes sous Apache, autoindex off sous Nginx).

Éviter de divulguer des chemins internes ou sensibles dans robots.txt, car ce fichier n'a aucune valeur de sécurité.

d5eec3ec36cf80dce44a896f961c1831a05526ec215693c8f2c39543497d4466



Fuite d'identifiants – htpasswd root

Accès au mot de passe root via /whatever contenant le fichier .htpasswd.
Déchiffrage du hash MD5 → mot de passe obtenu : qwerty123@

Recommandation :
Restreindre totalement l'accès aux fichiers .ht*, via Apache ou Nginx :

```
<FilesMatch "^\.ht">
   Require all denied
</FilesMatch>
```

ou pour Nginx :

```
location ~ /\.ht {
   deny all;
}
```